

Manipulator6DOF

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 accel Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	6
3.1.2.1 x	6
3.1.2.2 y	6
3.1.2.3 z	6
3.2 angles Struct Reference	6
3.2.1 Detailed Description	7
3.2.2 Member Data Documentation	7
3.2.2.1 ang_1	7
3.2.2.2 ang_2	7
3.2.2.3 ang_3	8
3.2.2.4 ang_4	8
3.2.2.5 ang_5	8
3.2.2.6 ang_6	8
3.3 gyro Struct Reference	9
3.3.1 Detailed Description	9
3.3.2 Member Data Documentation	9
3.3.2.1 x	9
3.3.2.2 y	10
3.3.2.3 z	10
3.4 inputData Struct Reference	10
3.4.1 Detailed Description	11
3.4.2 Member Data Documentation	11
3.4.2.1 accelData	11
3.4.2.2 angleData	11
3.4.2.3 gyroData	12
3.4.2.4 magData	12
3.5 mag Struct Reference	12
3.5.1 Detailed Description	13
3.5.2 Member Data Documentation	13
3.5.2.1 x	13
3.5.2.2 y	13
3.5.2.3 z	13
4 File Documentation	15

4.1 inc/definitions.h File Reference	15
4.1.1 Macro Definition Documentation	17
4.1.1.1 ANGLE_MAX_VALUE	17
4.1.1.2 APP_TITLE	17
4.1.1.3 CHART_TIME_RANGE	18
4.1.1.4 IMU_VALUE_RANGE	18
4.1.1.5 LABEL_FONT_SIZE	18
4.1.1.6 LANG_CODE	18
4.1.1.7 LANG_FONT_SIZE	18
4.1.1.8 LEFT_LAYOUT_WEIGHT	19
4.1.1.9 MANIPULATOR_HEIGHT	19
4.1.1.10 MANIPULATOR_WIDTH	19
4.1.1.11 NUM_ANGLE_CHARTS	19
4.1.1.12 NUM_IMU_CHARTS	19
4.1.1.13 RIGHT_LAYOUT_WEIGHT	20
4.1.1.14 TITLE_FONT_SIZE	20
4.1.2 Variable Documentation	20
4.1.2.1 ANGLE_COLORS	20
4.1.2.2 IMU_AXIS_LABELS	20
4.1.2.3 IMU_COLORS	21
4.1.2.4 IMU_LABELS	21
4.2 inc/qtFunctions.h File Reference	21
4.2.1 Function Documentation	22
4.2.1.1 createAngleChart()	22
4.2.1.2 createFlagPlaceholder()	23
4.2.1.3 createIMUChart()	24
4.2.1.4 createManipulatorPlaceholder()	25
4.2.1.5 createRotatedText()	25
4.3 src/main.cpp File Reference	26
4.3.1 Detailed Description	27
4.3.2 Function Documentation	27
4.3.2.1 main()	27
4.3.3 Variable Documentation	30
4.3.3.1 dataQueue	30
4.4 src/qtFunctions.cpp File Reference	30
4.4.1 Function Documentation	30
4.4.1.1 createAngleChart()	30
4.4.1.2 createFlagPlaceholder()	31
4.4.1.3 createIMUChart()	32
4.4.1.4 createManipulatorPlaceholder()	33
4.4.1.5 createRotatedText()	34

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

accel	Dane z akcelerometru	5
angles	Struktura przechowująca wartości kątów	6
gyro	Dane z żyroskopu	9
inputData	Struktura przechowująca dane wejściowe dla manipulatora 6 DOF	10
mag	Dane z magnetometru	12

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

inc/definitions.h	15
inc/qtFunctions.h	21
src/main.cpp	
Aplikacja do wizualizacji i modelowania manipulatorem 6 DOF	26
src/qtFunctions.cpp	30

Chapter 3

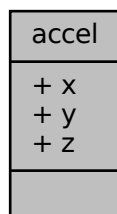
Class Documentation

3.1 accel Struct Reference

Dane z akcelerometru.

```
#include <definitions.h>
```

Collaboration diagram for accel:



Public Attributes

- float [x](#)
- float [y](#)
- float [z](#)

3.1.1 Detailed Description

Dane z akcelerometru.

Zawiera wartości przyspieszenia dla osi X, Y, Z. Jednostką miary jest m/s^2 .

Definition at line 114 of file definitions.h.

3.1.2 Member Data Documentation

3.1.2.1 x

```
float accel::x
```

Przyspieszenie w osi X

Definition at line 115 of file definitions.h.

3.1.2.2 y

```
float accel::y
```

Przyspieszenie w osi Y

Definition at line 116 of file definitions.h.

3.1.2.3 z

```
float accel::z
```

Przyspieszenie w osi Z

Definition at line 117 of file definitions.h.

The documentation for this struct was generated from the following file:

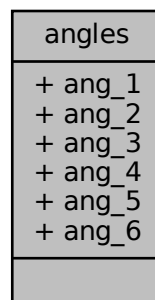
- [inc/definitions.h](#)

3.2 angles Struct Reference

Struktura przechowująca wartości kątów.

```
#include <definitions.h>
```

Collaboration diagram for angles:



Public Attributes

- float [ang_1](#)
- float [ang_2](#)
- float [ang_3](#)
- float [ang_4](#)
- float [ang_5](#)
- float [ang_6](#)

3.2.1 Detailed Description

Struktura przechowująca wartości kątów.

Struktura ta zawiera sześć zmiennych typu float, które reprezentują różne kąty. Każdy z tych kątów może być wykorzystywany w różnych obliczeniach związanych z geometrią lub ruchem.

Note

Kąty są przechowywane w jednostkach miary stopni.

Definition at line 100 of file definitions.h.

3.2.2 Member Data Documentation

3.2.2.1 [ang_1](#)

```
float angles::ang_1
```

Kąt 1

Definition at line 101 of file definitions.h.

3.2.2.2 [ang_2](#)

```
float angles::ang_2
```

Kąt 2

Definition at line 102 of file definitions.h.

3.2.2.3 ang_3

```
float angles::ang_3
```

Kąt 3

Definition at line 103 of file definitions.h.

3.2.2.4 ang_4

```
float angles::ang_4
```

Kąt 4

Definition at line 104 of file definitions.h.

3.2.2.5 ang_5

```
float angles::ang_5
```

Kąt 5

Definition at line 105 of file definitions.h.

3.2.2.6 ang_6

```
float angles::ang_6
```

Kąt 6

Definition at line 106 of file definitions.h.

The documentation for this struct was generated from the following file:

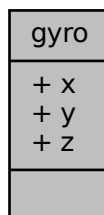
- [inc/definitions.h](#)

3.3 gyro Struct Reference

Dane z żyroskopu.

```
#include <definitions.h>
```

Collaboration diagram for gyro:



Public Attributes

- float [x](#)
- float [y](#)
- float [z](#)

3.3.1 Detailed Description

Dane z żyroskopu.

Zawiera wartości prędkości kątowej dla osi X, Y, Z. Jednostką miary jest stopień na sekundę (°/s).

Definition at line 125 of file definitions.h.

3.3.2 Member Data Documentation

3.3.2.1 x

```
float gyro::x
```

Prędkość kątowa w osi X

Definition at line 126 of file definitions.h.

3.3.2.2 y

```
float gyro::y
```

Prędkość kątowna w osi Y

Definition at line 127 of file definitions.h.

3.3.2.3 z

```
float gyro::z
```

Prędkość kątowna w osi Z

Definition at line 128 of file definitions.h.

The documentation for this struct was generated from the following file:

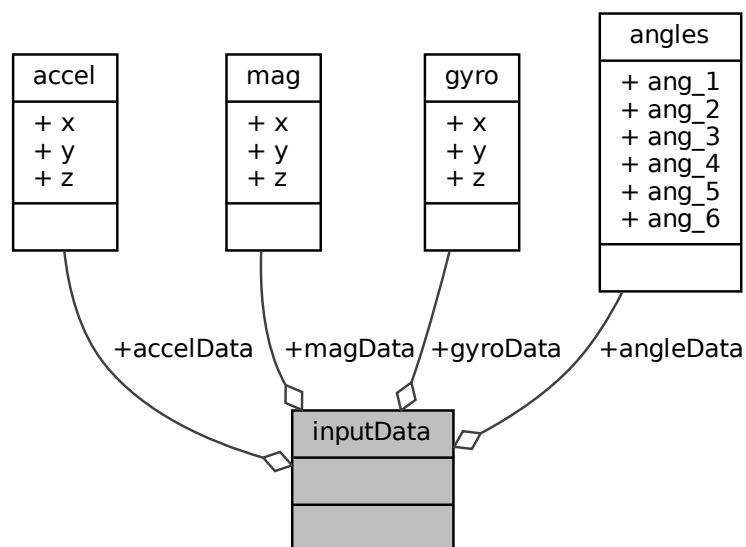
- [inc/definitions.h](#)

3.4 inputData Struct Reference

Struktura przechowująca dane wejściowe dla manipulatora 6 DOF.

```
#include <definitions.h>
```

Collaboration diagram for inputData:



Public Attributes

- [angles angleData](#)
Dane kątów manipulatora.
- [accel accelData](#)
Dane z akcelerometru.
- [gyro gyroData](#)
Dane z żyroskopu.
- [mag magData](#)
Dane z magnetometru.

3.4.1 Detailed Description

Struktura przechowująca dane wejściowe dla manipulatora 6 DOF.

Struktura zawiera dane z różnych czujników: kąty, akcelerometr, żyroskop, magnetometr. Dane te są przechowywane w kolejce typu `QQueue`, umożliwiając organizację i przetwarzanie danych wejściowych w czasie rzeczywistym.

Note

Kolejka pozwala na przechowywanie historycznych danych i ich późniejsze przetwarzanie.

Definition at line 151 of file definitions.h.

3.4.2 Member Data Documentation

3.4.2.1 accelData

```
accel inputData::accelData
```

Dane z akcelerometru.

Definition at line 153 of file definitions.h.

3.4.2.2 angleData

```
angles inputData::angleData
```

Dane kątów manipulatora.

Definition at line 152 of file definitions.h.

3.4.2.3 gyroData

```
gyro inputData::gyroData
```

Dane z żyroskopu.

Definition at line 154 of file definitions.h.

3.4.2.4 magData

```
mag inputData::magData
```

Dane z magnetometru.

Definition at line 155 of file definitions.h.

The documentation for this struct was generated from the following file:

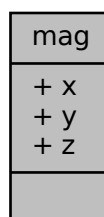
- [inc/definitions.h](#)

3.5 mag Struct Reference

Dane z magnetometru.

```
#include <definitions.h>
```

Collaboration diagram for mag:



Public Attributes

- float [x](#)
- float [y](#)
- float [z](#)

3.5.1 Detailed Description

Dane z magnetometru.

Zawiera wartości natężenia pola magnetycznego dla osi X, Y, Z. Jednostką miary jest uT (mikrotesla).

Definition at line 136 of file definitions.h.

3.5.2 Member Data Documentation

3.5.2.1 x

```
float mag::x
```

Natężenie pola magnetycznego w osi X

Definition at line 137 of file definitions.h.

3.5.2.2 y

```
float mag::y
```

Natężenie pola magnetycznego w osi Y

Definition at line 138 of file definitions.h.

3.5.2.3 z

```
float mag::z
```

Natężenie pola magnetycznego w osi Z

Definition at line 139 of file definitions.h.

The documentation for this struct was generated from the following file:

- [inc/definitions.h](#)

Chapter 4

File Documentation

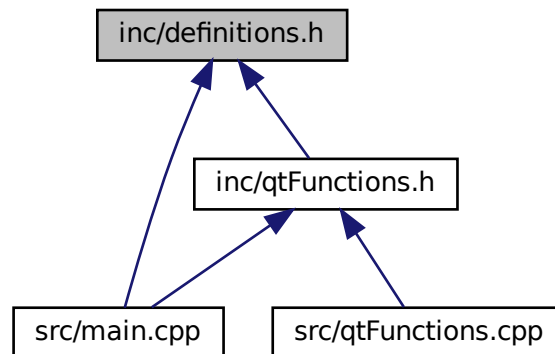
4.1 inc/definitions.h File Reference

```
#include <QApplication>
#include <QMainWindow>
#include <QHBoxLayout>
#include <QLabel>
#include <QChart>
#include <QChartView>
#include <QLineSeries>
#include <QValueAxis>
#include <QFont>
#include <QFrame>
#include <QDial>
#include <QPixmap>
#include <QGraphicsScene>
#include <QGraphicsView>
#include <QGraphicsPixmapItem>
#include <QPainter>
#include <QLineEdit>
#include <QDebug>
#include <QDir>
#include <QQueue>
```

Include dependency graph for definitions.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [angles](#)
Struktura przechowująca wartości kątów.
- struct [accel](#)
Dane z akcelerometru.
- struct [gyro](#)
Dane z żyroskopu.
- struct [mag](#)
Dane z magnetometru.
- struct [inputData](#)
Struktura przechowująca dane wejściowe dla manipulatora 6 DOF.

Macros

- #define [APP_TITLE](#) "Manipulator 6 DOF"
Stałe konfiguracyjne dla aplikacji.
- #define [LANG_CODE](#) "EN"
Domyślny kod języka.
- #define [LEFT_LAYOUT_WEIGHT](#) 7
Waga lewego panelu w układzie.
- #define [RIGHT_LAYOUT_WEIGHT](#) 3
Waga prawego panelu w układzie.
- #define [NUM_ANGLE_CHARTS](#) 6
Liczba wykresów kątów (potencjometrów)
- #define [NUM_IMU_CHARTS](#) 3
Liczba wykresów IMU.
- #define [CHART_TIME_RANGE](#) 30
Zakres czasu dla osi X (w sekundach)
- #define [ANGLE_MAX_VALUE](#) 180

- Maksymalna wartość kąta (w stopniach)*
 - `#define IMU_VALUE_RANGE` 20
 - Zakres wartości IMU (od -IMU_VALUE_RANGE do +IMU_VALUE_RANGE)*
- Rozmiar czcionki tytułu.*
 - `#define LABEL_FONT_SIZE` 12
 - Rozmiar czcionki etykiet.*
- Rozmiar czcionki dla oznaczenia języka.*
 - `#define LANG_FONT_SIZE` 12
 - Szerokość placeholdera manipulatora.*
- Wysokość placeholdera manipulatora.*
 - `#define MANIPULATOR_HEIGHT` 350

Variables

- `const QColor ANGLE_COLORS []`
 - Tablice z kolorami dla wykresów.*
- `const QColor IMU_COLORS []`
- `const QString IMU_LABELS []`
- `const QString IMU_AXIS_LABELS []`
 - Etykiety dla osi IMU.*

4.1.1 Macro Definition Documentation

4.1.1.1 ANGLE_MAX_VALUE

```
#define ANGLE_MAX_VALUE 180
```

Maksymalna wartość kąta (w stopniach)

Definition at line 44 of file definitions.h.

4.1.1.2 APP_TITLE

```
#define APP_TITLE "Manipulator 6 DOF"
```

Stałe konfiguracyjne dla aplikacji.

Tytuł aplikacji

Definition at line 33 of file definitions.h.

4.1.1.3 CHART_TIME_RANGE

```
#define CHART_TIME_RANGE 30
```

Zakres czasu dla osi X (w sekundach)

Definition at line 43 of file definitions.h.

4.1.1.4 IMU_VALUE_RANGE

```
#define IMU_VALUE_RANGE 20
```

Zakres wartości IMU (od -IMU_VALUE_RANGE do +IMU_VALUE_RANGE)

Definition at line 45 of file definitions.h.

4.1.1.5 LABEL_FONT_SIZE

```
#define LABEL_FONT_SIZE 12
```

Rozmiar czcionki etykiet.

Definition at line 49 of file definitions.h.

4.1.1.6 LANG_CODE

```
#define LANG_CODE "EN"
```

Domyślny kod języka.

Definition at line 34 of file definitions.h.

4.1.1.7 LANG_FONT_SIZE

```
#define LANG_FONT_SIZE 12
```

Rozmiar czcionki dla oznaczenia języka.

Definition at line 50 of file definitions.h.

4.1.1.8 LEFT_LAYOUT_WEIGHT

```
#define LEFT_LAYOUT_WEIGHT 7
```

Waga lewego panelu w układzie.

Definition at line 37 of file definitions.h.

4.1.1.9 MANIPULATOR_HEIGHT

```
#define MANIPULATOR_HEIGHT 350
```

Wysokość placeholdera manipulatora.

Definition at line 54 of file definitions.h.

4.1.1.10 MANIPULATOR_WIDTH

```
#define MANIPULATOR_WIDTH 350
```

Szerokość placeholdera manipulatora.

Definition at line 53 of file definitions.h.

4.1.1.11 NUM_ANGLE_CHARTS

```
#define NUM_ANGLE_CHARTS 6
```

Liczba wykresów kątów (potencjometrów)

Definition at line 41 of file definitions.h.

4.1.1.12 NUM_IMU_CHARTS

```
#define NUM_IMU_CHARTS 3
```

Liczba wykresów IMU.

Definition at line 42 of file definitions.h.

4.1.1.13 RIGHT_LAYOUT_WEIGHT

```
#define RIGHT_LAYOUT_WEIGHT 3
```

Waga prawego panelu w układzie.

Definition at line 38 of file definitions.h.

4.1.1.14 TITLE_FONT_SIZE

```
#define TITLE_FONT_SIZE 24
```

Rozmiar czcionki tytułu.

Definition at line 48 of file definitions.h.

4.1.2 Variable Documentation

4.1.2.1 ANGLE_COLORS

```
const QColor ANGLE_COLORS[ ]
```

Initial value:

```
= {  
    QColor(255, 0, 0),  
    QColor(0, 0, 255),  
    QColor(0, 200, 0),  
    QColor(255, 200, 0),  
    QColor(200, 0, 200),  
    QColor(150, 75, 75)  
}
```

Tablice z kolorami dla wykresów.

Definition at line 59 of file definitions.h.

4.1.2.2 IMU_AXIS_LABELS

```
const QString IMU_AXIS_LABELS[ ]
```

Initial value:

```
= {  
    "X axis",  
    "Y axis",  
    "Z axis"  
}
```

Etykiety dla osi IMU.

Definition at line 85 of file definitions.h.

Functions

- QT_CHARTS_USE_NAMESPACE QPixmap [createRotatedText](#) (const QString &text, const QFont &font, int width, int height)
Funkcja tworząca obrócony tekst w etykiecie.
- QChartView * [createAngleChart](#) (int colorIndex)
Funkcja tworząca wykres kąta.
- QChartView * [createIMUChart](#) (int axisIndex)
Funkcja tworząca wykres IMU.
- QPixmap [createFlagPlaceholder](#) (const QString &text, const QColor &color)
Funkcja tworząca placeholder flagi.
- QPixmap [createManipulatorPlaceholder](#) ()
Funkcja tworząca placeholder manipulatora jeśli brak obrazu.

4.2.1 Function Documentation

4.2.1.1 createAngleChart()

```
QChartView* createAngleChart (
    int colorIndex )
```

Funkcja tworząca wykres kąta.

Parameters

<i>colorIndex</i>	Indeks koloru z tablicy ANGLE_COLORS
-------------------	--------------------------------------

Returns

QChartView z przygotowanym wykresem

Definition at line 19 of file qtFunctions.cpp.

```
19 {
20     QChart *chart = new QChart();
21     chart->legend()->hide();
22     chart->setMargins(QMargins(5, 5, 5, 5));
23     chart->setBackgroundBrush(QBrush(Qt::white));
24     chart->setTitle("");
25
26     QLineSeries *series = new QLineSeries();
27
28     QPen pen(ANGLE_COLORS[colorIndex]);
29     pen.setWidth(2);
30     series->setPen(pen);
31
32     chart->addSeries(series);
33
34     QValueAxis *axisX = new QValueAxis();
35     axisX->setRange(0, CHART_TIME_RANGE);
36     axisX->setGridLineVisible(true);
37     axisX->setTickCount(7);
38     axisX->setLabelText("");
39     axisX->setLabelsVisible(false);
40
41     QValueAxis *axisY = new QValueAxis();
42     axisY->setRange(0, ANGLE_MAX_VALUE);
43     axisY->setGridLineVisible(true);
```

```

44     axisY->setTickCount(5);
45     axisY->setTitleText("");
46     axisY->setLabelsVisible(false);
47
48     chart->setAxisX(axisX, series);
49     chart->setAxisY(axisY, series);
50
51     QChartView *chartView = new QChartView(chart);
52     chartView->setRenderHint(QPainter::Antialiasing);
53     chartView->setMinimumHeight(80);
54
55     return chartView;
56 }

```

Here is the caller graph for this function:



4.2.1.2 createFlagPlaceholder()

```

QPixmap createFlagPlaceholder (
    const QString & text,
    const QColor & color )

```

Funkcja tworząca placeholder flagi.

Parameters

<i>text</i>	Tekst na placeholderze flagi
<i>color</i>	Kolor tła flagi

Returns

QPixmap z flagą

Definition at line 95 of file qtFunctions.cpp.

```

95
96     QPixmap flagPixmap(30, 20);
97     flagPixmap.fill(color);
98
99     QPainter painter(&flagPixmap);
100     painter.setPen(Qt::black);
101     painter.drawText(flagPixmap.rect(), Qt::AlignCenter, text);
102     painter.drawRect(0, 0, flagPixmap.width() - 1, flagPixmap.height() - 1);
103     painter.end();
104
105     return flagPixmap;
106 }

```

4.2.1.3 createIMUChart()

```
QChartView* createIMUChart (
    int axisIndex )
```

Funkcja tworząca wykres IMU.

Parameters

<i>axisIndex</i>	Indeks osi (0-X, 1-Y, 2-Z)
------------------	----------------------------

Returns

QChartView z przygotowanym wykresem IMU

Definition at line 58 of file qtFunctions.cpp.

```
58 {
59     QChart *chart = new QChart();
60     chart->legend()->hide();
61     chart->setMargins(QMargins(5, 5, 5, 5));
62     chart->setBackgroundBrush(QBrush(Qt::white));
63     chart->setTitle("");
64
65     QLineSeries *series = new QLineSeries();
66     QPen pen(IMU_COLORS[axisIndex]);
67     pen.setWidth(2);
68     series->setPen(pen);
69     chart->addSeries(series);
70
71     QValueAxis *axisX = new QValueAxis();
72     axisX->setRange(0, CHART_TIME_RANGE);
73     axisX->setGridLineVisible(true);
74     axisX->setTickCount(7);
75     axisX->setTitleText("");
76     axisX->setLabelsVisible(false);
77
78     QValueAxis *axisY = new QValueAxis();
79     axisY->setRange(-IMU_VALUE_RANGE, IMU_VALUE_RANGE);
80     axisY->setGridLineVisible(true);
81     axisY->setTickCount(5);
82     axisY->setTitleText("");
83     axisY->setLabelsVisible(false);
84
85     chart->setAxisX(axisX, series);
86     chart->setAxisY(axisY, series);
87
88     QChartView *chartView = new QChartView(chart);
89     chartView->setRenderHint(QPainter::Antialiasing);
90     chartView->setMinimumHeight(80);
91
92     return chartView;
93 }
```

Here is the caller graph for this function:



4.2.1.4 createManipulatorPlaceholder()

QPixmap createManipulatorPlaceholder ()

Funkcja tworząca placeholder manipulatora jeśli brak obrazu.

Returns

QPixmap z placeholderem

Definition at line 108 of file qtFunctions.cpp.

```

108 {
109     QPixmap placeholder(MANIPULATOR_WIDTH, MANIPULATOR_HEIGHT);
110     placeholder.fill(Qt::white);
111
112     QPainter painter(&placeholder);
113     painter.setPen(QPen(Qt::black, 2));
114
115     int centerX = MANIPULATOR_WIDTH / 2;
116     int baseY = MANIPULATOR_HEIGHT - 50;
117
118     painter.drawRect(centerX - 50, baseY, 100, 30);
119     painter.drawLine(centerX, baseY, centerX, baseY - 100);
120     painter.drawLine(centerX, baseY - 100, centerX + 120, baseY - 150);
121     painter.drawLine(centerX + 120, baseY - 150, centerX + 150, baseY - 250);
122     painter.drawEllipse(centerX + 140, baseY - 270, 20, 20);
123
124     painter.drawText(10, 20, "Manipulator 6 DOF (placeholder)");
125
126     painter.end();
127     return placeholder;
128 }
```

4.2.1.5 createRotatedText()

QT_CHARTS_USE_NAMESPACE QPixmap createRotatedText (

```

    const QString & text,
    const QFont & font,
    int width,
    int height )
```

Funkcja tworząca obrocony tekst w etykiecie.

Parameters

<i>text</i>	Tekst do obrócenia
<i>font</i>	Czcionka dla tekstu
<i>width</i>	Szerokość wynikowego pixmapy
<i>height</i>	Wysokość wynikowej pixmapy

Returns

QPixmap z obróconym tekstem

Definition at line 3 of file qtFunctions.cpp.

```

3 {
4     QTransform transform;
5     transform.rotate(-90);
```


4.3.1 Detailed Description

Aplikacja do wizualizacji i modelowania manipulatorem 6 DOF.

Author

Michał Markuzel

Date

2025-04-22

Program umożliwia monitorowanie kątów przegubów manipulatora oraz danych z IMU. Aplikacja zawiera wykresy dla 6 potencjometrów oraz 3 zestawy wykresów dla danych z IMU. Dodatkowo przygotowane jest miejsce na wizualizację 3D manipulatora.

4.3.2 Function Documentation

4.3.2.1 main()

```
int main (
    int argc,
    char * argv[] )
```

Główna funkcja programu.

Parameters

<i>argc</i>	Liczba argumentów wiersza poleceń
<i>argv</i>	Tablica argumentów wiersza poleceń

Returns

Kod zakończenia programu

Definition at line 30 of file main.cpp.

```
30 {
31     QApplication app(argc, argv);
32
33     // Tworzenie głównego okna
34     QMainWindow window;
35     window.setWindowTitle(APP_TITLE);
36
37     // Główny widżet
38     QWidget *centralWidget = new QWidget(&window);
39     window.setCentralWidget(centralWidget);
40
41     // Główny układ
42     QHBoxLayout *mainLayout = new QHBoxLayout(centralWidget);
43
44     // Część lewa - wykresy i potencjometry
45     QVBoxLayout *leftLayout = new QVBoxLayout();
46 }
```

```

47 // Tworzymy siatkę dla lewej części
48 QGridLayout *chartsGridLayout = new QGridLayout();
49
50 // Przygotowanie czcionki dla etykiet
51 QFont labelFont;
52 labelFont.setBold(true);
53 labelFont.setPointSize(LABEL_FONT_SIZE);
54
55 // Dodajemy pionowy napis ANGLES
56 QLabel *anglesLabel = new QLabel("ANGLES");
57 anglesLabel->setFixedWidth(40);
58 anglesLabel->setFont(labelFont);
59 anglesLabel->setPixmap(createRotatedText("ANGLES", labelFont, 50, 200));
60
61 chartsGridLayout->addWidget(anglesLabel, 0, 0, NUM_ANGLE_CHARTS, 1);
62
63 // Tworzenie wykresów dla potencjometrów
64 for (int i = 0; i < NUM_ANGLE_CHARTS; i++) {
65     // Etykieta dla angle
66     QLabel *angleLabel = new QLabel("angle");
67     angleLabel->setFixedWidth(40);
68     QString label = QString("angle_%1").arg(i + 1);
69     angleLabel->setPixmap(createRotatedText(label, labelFont, 40, 100));
70
71     // Tworzenie wykresu
72     QChartView *chartView = createAngleChart(i);
73
74     // Potencjometr
75     QDial *dial = new QDial();
76     dial->setFixedSize(60, 60);
77
78     // Dodajemy wykres i potencjometr do siatki
79     chartsGridLayout->addWidget(angleLabel, i, 1);
80     chartsGridLayout->addWidget(chartView, i, 2);
81     chartsGridLayout->addWidget(dial, i, 3);
82 }
83
84 // Sekcja IMU
85 QLabel *imuLabel = new QLabel("IMU");
86 imuLabel->setFixedWidth(40);
87 imuLabel->setPixmap(createRotatedText("IMU", labelFont, 50, 150));
88
89 chartsGridLayout->addWidget(imuLabel, NUM_ANGLE_CHARTS, 0, NUM_IMU_CHARTS, 1);
90
91 // Tworzymy wykresy dla IMU - z tym samym wyglądem jak dla angles
92 for (int i = 0; i < NUM_IMU_CHARTS; i++) {
93     // Etykieta dla osi IMU (X, Y, Z)
94     QLabel *axisLabel = new QLabel(IMU_AXIS_LABELS[i]);
95     axisLabel->setFixedWidth(40);
96     axisLabel->setPixmap(createRotatedText(IMU_AXIS_LABELS[i], labelFont, 40, 80));
97
98     // Tworzenie wykresu IMU z tym samym stylem co wykresy angle
99     QChartView *chartView = createIMUChart(i);
100
101     // Legenda pod wykresem
102     QHBoxLayout *legendLayout = new QHBoxLayout();
103
104     for (int j = 0; j < 4; j++) {
105         QLabel *legendLabel = new QLabel(" " + IMU_LABELS[j]);
106         QString styleSheet = QString("color: %1; font-size: 8pt;").arg(IMU_COLORS[j].name());
107         legendLabel->setStyleSheet(styleSheet);
108         legendLabel->setAlignment(Qt::AlignCenter);
109         legendLayout->addWidget(legendLabel);
110     }
111
112     QVBoxLayout *imuChartLayout = new QVBoxLayout();
113     imuChartLayout->addWidget(chartView);
114     imuChartLayout->addLayout(legendLayout);
115
116     chartsGridLayout->addWidget(axisLabel, i + NUM_ANGLE_CHARTS, 1);
117     chartsGridLayout->addLayout(imuChartLayout, i + NUM_ANGLE_CHARTS, 2);
118 }
119
120 // Ustawiamy rozciąganie wierszy
121 for (int i = 0; i < NUM_ANGLE_CHARTS + NUM_IMU_CHARTS; i++) {
122     chartsGridLayout->setRowStretch(i, 1);
123 }
124
125 // Dodajemy siatkę wykresów do lewego layoutu
126 leftLayout->addLayout(chartsGridLayout);
127
128 // Część prawa - tytuł i wizualizacja manipulatora
129 QVBoxLayout *rightLayout = new QVBoxLayout();
130
131 // Tytuł
132 QLabel *titleLabel = new QLabel(APP_TITLE);
133 QFont titleFont = titleLabel->font();

```

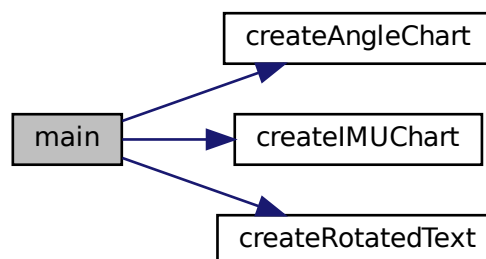


```

134     titleFont.setBold(true);
135     titleFont.setPointSize(TITLE_FONT_SIZE);
136     titleLabel->setFont(titleFont);
137     titleLabel->setAlignment(Qt::AlignCenter);
138     rightLayout->addWidget(titleLabel);
139
140     // Miejsce na wizualizację manipulatora
141     QGraphicsScene *scene = new QGraphicsScene();
142     QGraphicsView *manipulatorView = new QGraphicsView(scene);
143     manipulatorView->setMinimumSize(400, 400);
144
145     // Ladowanie obrazu manipulatora
146     QPixmap manipulatorPixmap;
147     manipulatorPixmap = QPixmap(":/icons/icons/manipulator_6dof.png");
148
149     QGraphicsPixmapItem *manipulatorItem = scene->addPixmap(manipulatorPixmap);
150     manipulatorItem->setPos(0, 0);
151
152     rightLayout->addWidget(manipulatorView);
153
154     // Ikony flag
155     QHBoxLayout *flagsLayout = new QHBoxLayout();
156
157     // Ladowanie ikon flag
158     QPixmap ukFlagPixmap;
159     QPixmap plFlagPixmap;
160
161     ukFlagPixmap = QPixmap(":/icons/icons/uk.png");
162     plFlagPixmap = QPixmap(":/icons/icons/pl.png");
163
164
165     // Tworzenie etykiet z ikonami flag
166     QLabel *ukFlagLabel = new QLabel();
167     ukFlagLabel->setPixmap(ukFlagPixmap.scaled(30, 20, Qt::KeepAspectRatio));
168     ukFlagLabel->setToolTip("English");
169     ukFlagLabel->setCursor(Qt::PointingHandCursor);
170
171     QLabel *plFlagLabel = new QLabel();
172     plFlagLabel->setPixmap(plFlagPixmap.scaled(30, 20, Qt::KeepAspectRatio));
173     plFlagLabel->setToolTip("Polski");
174     plFlagLabel->setCursor(Qt::PointingHandCursor);
175
176     flagsLayout->addStretch();
177     flagsLayout->addWidget(ukFlagLabel);
178     flagsLayout->addSpacing(10);
179     flagsLayout->addWidget(plFlagLabel);
180     flagsLayout->addSpacing(10);
181
182     rightLayout->addLayout(flagsLayout);
183     rightLayout->addStretch();
184
185     // Dodanie lewy i prawego layout'u do głównego layoutu
186     mainLayout->addLayout(leftLayout, LEFT_LAYOUT_WEIGHT);
187     mainLayout->addLayout(rightLayout, RIGHT_LAYOUT_WEIGHT);
188
189     // Ustawiamy okno jako zmaksymalizowane
190     window.showMaximized();
191
192     return app.exec();
193 }

```

Here is the call graph for this function:



Parameters

<i>colorIndex</i>	Indeks koloru z tablicy ANGLE_COLORS
-------------------	--------------------------------------

Returns

QChartView z przygotowanym wykresem

Definition at line 19 of file qtFunctions.cpp.

```

19      {
20          QChart *chart = new QChart();
21          chart->legend()->hide();
22          chart->setMargins(QMargins(5, 5, 5, 5));
23          chart->setBackgroundBrush(QBrush(Qt::white));
24          chart->setTitle("");
25
26          QLineSeries *series = new QLineSeries();
27
28          QPen pen(ANGLE_COLORS[colorIndex]);
29          pen.setWidth(2);
30          series->setPen(pen);
31
32          chart->addSeries(series);
33
34          QValueAxis *axisX = new QValueAxis();
35          axisX->setRange(0, CHART_TIME_RANGE);
36          axisX->setGridLineVisible(true);
37          axisX->setTickCount(7);
38          axisX->setTitleText("");
39          axisX->setLabelsVisible(false);
40
41          QValueAxis *axisY = new QValueAxis();
42          axisY->setRange(0, ANGLE_MAX_VALUE);
43          axisY->setGridLineVisible(true);
44          axisY->setTickCount(5);
45          axisY->setTitleText("");
46          axisY->setLabelsVisible(false);
47
48          chart->setAxisX(axisX, series);
49          chart->setAxisY(axisY, series);
50
51          QChartView *chartView = new QChartView(chart);
52          chartView->setRenderHint(QPainter::Antialiasing);
53          chartView->setMinimumHeight(80);
54
55          return chartView;
56      }

```

Here is the caller graph for this function:



4.4.1.2 createFlagPlaceholder()

```

QPixmap createFlagPlaceholder (
    const QString & text,
    const QColor & color )

```

Funkcja tworząca placeholder flagi.

Parameters

<i>text</i>	Tekst na placeholderze flagi
<i>color</i>	Kolor tła flagi

Returns

QPixmap z flagą

Definition at line 95 of file qtFunctions.cpp.

```

95                                     {
96     QPixmap flagPixmap(30, 20);
97     flagPixmap.fill(color);
98
99     QPainter painter(&flagPixmap);
100    painter.setPen(Qt::black);
101    painter.drawText(flagPixmap.rect(), Qt::AlignCenter, text);
102    painter.drawRect(0, 0, flagPixmap.width() - 1, flagPixmap.height() - 1);
103    painter.end();
104
105    return flagPixmap;
106 }
```

4.4.1.3 createIMUChart()

```

QChartView* createIMUChart (
    int axisIndex )
```

Funkcja tworząca wykres IMU.

Parameters

<i>axisIndex</i>	Indeks osi (0-X, 1-Y, 2-Z)
------------------	----------------------------

Returns

QChartView z przygotowanym wykresem IMU

Definition at line 58 of file qtFunctions.cpp.

```

58                                     {
59     QChart *chart = new QChart();
60     chart->legend()->hide();
61     chart->setMargins(QMargins(5, 5, 5, 5));
62     chart->setBackgroundBrush(QBrush(Qt::white));
63     chart->setTitle("");
64
65     QLineSeries *series = new QLineSeries();
66     QPen pen(IMU_COLORS[axisIndex]);
67     pen.setWidth(2);
68     series->setPen(pen);
69     chart->addSeries(series);
70
71     QValueAxis *axisX = new QValueAxis();
72     axisX->setRange(0, CHART_TIME_RANGE);
73     axisX->setGridLineVisible(true);
74     axisX->setTickCount(7);
75     axisX->setTitleText("");
76     axisX->setLabelsVisible(false);
77
78     QValueAxis *axisY = new QValueAxis();
79     axisY->setRange(-IMU_VALUE_RANGE, IMU_VALUE_RANGE);
```

```

80     axisY->setGridLineVisible(true);
81     axisY->setTickCount(5);
82     axisY->setTitleText("");
83     axisY->setLabelsVisible(false);
84
85     chart->setAxisX(axisX, series);
86     chart->setAxisY(axisY, series);
87
88     QChartView *chartView = new QChartView(chart);
89     chartView->setRenderHint(QPainter::Antialiasing);
90     chartView->setMinimumHeight(80);
91
92     return chartView;
93 }

```

Here is the caller graph for this function:



4.4.1.4 createManipulatorPlaceholder()

QPixmap createManipulatorPlaceholder ()

Funkcja tworząca placeholder manipulatora jeśli brak obrazu.

Returns

QPixmap z placeholderem

Definition at line 108 of file qtFunctions.cpp.

```

108     {
109         QPixmap placeholder(MANIPULATOR_WIDTH, MANIPULATOR_HEIGHT);
110         placeholder.fill(Qt::white);
111
112         QPainter painter(&placeholder);
113         painter.setPen(QPen(Qt::black, 2));
114
115         int centerX = MANIPULATOR_WIDTH / 2;
116         int baseY = MANIPULATOR_HEIGHT - 50;
117
118         painter.drawRect(centerX - 50, baseY, 100, 30);
119         painter.drawLine(centerX, baseY, centerX, baseY - 100);
120         painter.drawLine(centerX, baseY - 100, centerX + 120, baseY - 150);
121         painter.drawLine(centerX + 120, baseY - 150, centerX + 150, baseY - 250);
122         painter.drawEllipse(centerX + 140, baseY - 270, 20, 20);
123
124         painter.drawText(10, 20, "Manipulator 6 DOF (placeholder)");
125
126         painter.end();
127         return placeholder;
128     }

```

4.4.1.5 createRotatedText()

```
QPixmap createRotatedText (
    const QString & text,
    const QFont & font,
    int width,
    int height )
```

Funkcja tworząca obrócony tekst w etykiecie.

Parameters

<i>text</i>	Tekst do obrócenia
<i>font</i>	Czcionka dla tekstu
<i>width</i>	Szerokość wynikowego pixmapy
<i>height</i>	Wysokość wynikowej pixmapy

Returns

QPixmap z obróconym tekstem

Definition at line 3 of file qtFunctions.cpp.

```
3
4     QTransform transform;
5     transform.rotate(-90);
6
7     QPixmap pixmap(width, height);
8     pixmap.fill(Qt::transparent);
9
10    QPainter painter(&pixmap);
11    painter.setTransform(transform);
12    painter.setFont(font);
13    painter.drawText(QRect(-height, 0, height, width), Qt::AlignCenter, text);
14    painter.end();
15
16    return pixmap;
17 }
```

Here is the caller graph for this function:

